Lecture 7: Induction, Simplification, and Software Verification

Stefano M. Nicoletti

1 Lecture 7: Induction, Simplification, and Software Verification

1.1 Aim of the lecture

We continue from the first induction proof in Lecture 6. We prove further properties of addition, introduce `simp`, and combine induction, rewriting, and simplification. We then apply the same tools to the verification of a recursive algorithm on lists.

1.2 Definitions and available results

We continue to use the natural numbers, addition, and doubling defined in Lecture 6:

`Natural` has the constructors `zero` and `succ`. The function `add (m n : Natural)` recurses on its second argument, while `double (n : Natural)` recurses on its only argument.

The argument examined by a definition determines when Lean can compute. The definition of `add` can select a branch when it knows the form of the second argument. Thus, `add m zero` and `add m (succ n)` reduce directly. By contrast, `add zero n` requires induction while `n` remains an unknown variable.

The computational equations and the final result from Lecture 6 are available:

```
open Natural

theorem lecture07_add_zero_right (m : Natural) : add m zero = m := by
  rfl
```

```
open Natural

theorem lecture07_add_succ_right (m n : Natural) :
    add m (succ n) = succ (add m n) := by
  rfl
```

```
open Natural

theorem lecture07_double_succ (n : Natural) :
```

```
    double (succ n) = succ (succ (double n)) := by
  rfl
```

```
open Natural

theorem lecture07_zero_add (n : Natural) : add zero n = n := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      unfold add
      rewrite [ih]
      rfl
```

The first three results state the computational behaviour of the definitions. The last is a general theorem proved by induction. This distinction remains important throughout the lecture: some transformations are computations, whereas others use previously proved results.

1.3 A second induction proof

We now prove that a successor in the first argument can be moved outside the addition:

`add (succ m) n = succ (add m n).`

The visible successor may suggest induction on `m`. The definition tells us otherwise. `add` examines its second argument, and that position contains `n` in both additions. The unknown form of `n` prevents the definition from choosing between its `zero` and `succ` branches, so we induct on `n`.

```
open Natural

theorem lecture07_succ_add (m n : Natural) :
    add (succ m) n = succ (add m n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      unfold add
      rewrite [ih]
      rfl
```

In the base case, replacing `n` with `zero` lets both sides reduce to the same expression. In the induction step, `unfold add` opens both additions for the case `succ n`. The remaining goal contains the same equality for `n`, which is exactly the induction hypothesis `ih`. Rewriting with `ih` makes both sides identical.

The induction variable therefore follows the computation. We look for the variable in the argument examined by the recursive definition and check that the induction hypothesis will rewrite the recursive call produced in the successor case.

1.4 Associativity, commutativity, and ‘simp‘

Associativity states that grouping three addends in different ways does not change the result. On the left, `c` is the second argument of the outer addition. On the right, it is the second argument of `add b c`, which in turn controls the outer computation. We therefore induct on `c`.

```
open Natural

theorem lecture07_add_assoc (a b c : Natural) :
    add (add a b) c = add a (add b c) := by
  induction c with
  | zero =>
      rfl
  | succ c ih =>
      rw [lecture07_add_succ_right]
      rw [lecture07_add_succ_right]
      rw [lecture07_add_succ_right]
      rw [ih]
```

In the base case, additions with `zero` as their second argument reduce, and `rfl` closes the goal. In the successor case, three rewrites with `lecture07_add_succ_right` expose the recursive additions on both sides. The remaining equality is the induction hypothesis under `succ`, so `rw [ih]` completes the proof.

Commutativity states that exchanging the two addends does not change the result. It does not follow from computation alone because `add` treats its two arguments asymmetrically: it examines the second argument and leaves the first unchanged. We induct on `b`. In the base case, the left side computes while the right side uses `lecture07_zero_add`. In the successor case, `lecture07_add_succ_right` handles the successor on the right and `lecture07_succ_add` handles the successor that appears in the first argument after the exchange.

```
open Natural

theorem lecture07_add_comm (a b : Natural) :
    add a b = add b a := by
  induction b with
  | zero =>
      rw [lecture07_zero_add]
      rfl
  | succ b ih =>
      rw [lecture07_add_succ_right]
      rw [lecture07_succ_add]
      rw [ih]
```

This proof explains why small supporting lemmas matter. The definition gives the equations for the second argument directly. The theorems `zero_add` and `succ_add` provide the corresponding forms for the first argument. Commutativity combines these two views.

When a proof requires many elementary rewrites, we can use the simplifier. `simp` applies equations oriented towards simpler forms and performs basic computations that Lean recognizes directly. It does not search arbitrarily for a proof. It uses a collection of rules called the *simp-set*.

We compare three uses of the same theorem. `rw` performs a specific rewrite chosen by us. `simp only [lemmas]` uses only the listed lemmas, together with the simplifier's essential reductions. `simp [lemma]` uses the standard rules and adds `lemma` for that command.

```
open Natural

theorem lecture07_zero_add_with_rw (n : Natural) : add zero n = n := by
  rw [lecture07_zero_add]
```

The first theorem names the exact rewrite. The second asks the simplifier to work with one declared rule. The third adds the theorem to the simp-set for a single invocation. All three prove the same statement, but they provide different degrees of control and automation.

```
open Natural

theorem lecture07_zero_add_with_simp_only (n : Natural) : add zero n = n := by
  simp only [lecture07_zero_add]
```

```
open Natural

theorem lecture07_zero_add_with_simp (n : Natural) : add zero n = n := by
  simp [lecture07_zero_add]
```

```
open Natural

section LocalSimplification

attribute [local simp] lecture07_zero_add lecture07_succ_add

theorem lecture07_succ_add_with_simp (m n : Natural) :
    add (succ m) n = succ (add m n) := by
  simp

end LocalSimplification
```

The `local simp` attribute applies only inside the section. It lets us extend the simp-set without changing every later proof in the module. Inside the section, `simp` finds `lecture07_succ_add` automatically, so the theorem need not be named in the tactic call.

```
open Natural

@[simp] theorem lecture07_double_zero :
```

```
    double zero = zero := by
  rfl
```

`@[simp]` registers a rule for the rest of the module. We use it for a canonical equation such as `double zero = zero`. Associativity and commutativity are not appropriate simp rules because they only move parentheses or swap arguments.

Rule selection matters. A good simplification rule predictably moves an expression towards a shorter or canonical form. Associativity would require a global choice of grouping. Commutativity could keep exchanging arguments without producing an evidently simpler expression. We therefore apply these results explicitly where they are needed.

1.5 A final proof

We combine induction and rewriting to show that the double of a sum is the sum of the doubles. The subexpression that remains blocked is `add m n`, so we induct on `n`.

```
open Natural

theorem lecture07_double_add (m n : Natural) :
    double (add m n) = add (double m) (double n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      rw [lecture07_add_succ_right]
      rw [lecture07_double_succ]
      rw [lecture07_double_succ]
      rw [lecture07_add_succ_right]
      rw [lecture07_add_succ_right]
      rw [ih]
```

The base case follows from the definitions of `add` and `double`. In the successor case, the first rewrite moves the successor out of `add m (succ n)`. The two rewrites with `lecture07_double_succ` describe doubling a successor on both sides. The next two addition rewrites expose the same successor structure. Finally, `ih` replaces the double of the smaller sum by the sum of the doubles.

After checking each step, we can express the same proof with a controlled collection of simplification rules:

```
open Natural

theorem lecture07_double_add_short (m n : Natural) :
    double (add m n) = add (double m) (double n) := by
  induction n with
  | zero =>
      rfl
  | succ n ih =>
      simp only [lecture07_add_succ_right, lecture07_double_succ, ih]
```

The shorter version does not use a different mathematical argument. It uses the same equations but lets `simp only` choose their occurrences. The explicit list remains useful because it records which results justify the proof.

1.6 From induction to software verification

So far, we have used induction to prove properties of mathematical operations. The same method allows us to reason about programs. A program takes an input and produces an output. To verify one of its properties, we define the program precisely, state the expected behaviour, and prove that this behaviour holds for every possible input.

Running the program on examples remains useful, but it does not provide this general guarantee. A test can reveal an error on a particular input. A universal proof instead covers every value that can be constructed in the relevant type.

1.6.1 6.1 An inductive type for lists

We define a list without using the library's `List` type. The parameter `α` represents the type of the stored elements. The same definition can therefore be used for lists of natural numbers, strings, or other objects.

```
inductive CustomList α( : Type) where
  | empty
  | cons (head : α) (tail : CustomList α)

open CustomList
```

A `CustomList α` is built in two ways. `empty` contains no elements. `cons x xs` contains the element `x` in front of the list `xs`. The tail `xs` is itself a list and is the smaller input on which recursion and induction can proceed.

The parameter `α` makes the definition polymorphic. We do not define a separate list type for every kind of element. `CustomList Natural` is the type of lists of natural numbers, whereas `CustomList String` would be the type of lists of strings. Both use the same constructors.

For example, we can construct a list containing three natural numbers:

```
def exampleList : CustomList Natural :=
  .cons .zero
    (.cons (.succ .zero)
      (.cons (.succ (.succ .zero)) .empty))
```

The braces in `{α : Type}` make `α` an implicit argument. When we write `append xs ys`, Lean infers the element type from the types of `xs` and `ys`. We therefore do not pass `α` manually at each call.

Computation follows the first list. If `xs` is `empty`, there are no elements to preserve and the result is `ys`. If `xs` is `cons x xs`, the result keeps `x` at its head and recursively appends the tail to `ys`. Using square brackets as illustrative notation:

```
append [zero, one] [two]
= zero :: append [one] [two]
= zero :: one :: append [] [two]
= [zero, one, two].
```

Here `[]` represents `empty`, and `x :: xs` represents `cons x xs`.

1.6.2 6.2 Append and reverse

To reverse a list, we need to place an element at the end of a result that has already been constructed. We first define append. The function examines the first list. If it is empty, append returns the second list. In the `cons` case, it retains the head and recursively appends the tail.

```
def append α{ : Type} (xs ys : CustomList α) : CustomList α :=
  match xs with
  | .empty => ys
  | .cons x xs => .cons x (append xs ys)
```

`reverse` is also polymorphic. In the `cons x xs` case, the recursive call first reverses the tail. The singleton list containing `x` is then appended at the end. For `[zero, one]`, the computation is:

```
reverse [zero, one]
= append (reverse [one]) [zero]
= append [one] [zero]
= [one, zero].
```

The `reverse` algorithm follows the same inductive structure. The empty list remains empty. In the `cons x xs` case, it reverses the tail `xs` and places `x` at the end using `append`.

```
def reverse α{ : Type} (xs : CustomList α) : CustomList α :=
  match xs with
  | .empty => .empty
  | .cons x xs => append (reverse xs) (.cons x .empty)
```

The recursive call receives the tail, which contains one fewer `cons` constructor. The definition therefore eventually reaches the `empty` case.

1.6.3 6.3 Stating a correctness property

One property of the algorithm is that reversing twice returns the original list:

```
lecture07_reverse_reverse {α : Type} (xs : CustomList α) :
  reverse (reverse xs) = xs
```

The statement is not restricted to `exampleList`. It must hold for every type `α` and every list `xs : CustomList α`. This is an important correctness property, although it is not by itself a complete specification of every aspect of reversal.

To prove this property, we first establish results about `append` and about the interaction between `reverse` and `append`. We then use those results in the final theorem.

1.6.4 6.4 Properties of append

Appending the empty list on the right leaves a list unchanged. Because `append` examines its first argument, we prove the result by induction on `xs`.

```
theorem lecture07_append_empty_right
    α{ : Type} (xs : CustomList α) :
    append xs .empty = xs := by
  induction xs with
  | empty =>
      rfl
  | cons x xs ih =>
      simp [append, ih]
```

In the `empty` case, `append empty empty` reduces directly to `empty`. In the `cons x xs` case, the definition retains `x` and leaves the result for the tail `xs`. The induction hypothesis resolves precisely that subexpression.

This equation removes an append with the empty list on the right and produces a simpler form. We can add it to the local simp-set:

```
attribute [local simp] lecture07_append_empty_right
```

We also need associativity of append. Again, the induction variable is `xs` because it occupies the position examined by the definition.

```
theorem lecture07_append_assoc
    α{ : Type} (xs ys zs : CustomList α) :
    append (append xs ys) zs = append xs (append ys zs) := by
  induction xs with
  | empty =>
      rfl
  | cons x xs ih =>
      simp [append, ih]
```

We again induct on `xs` because it is the first argument of the outer append on the left and the argument examined by the definition. In the `cons` case, both sides retain the same head `x`, and the induction hypothesis aligns the appended tails.

We do not add associativity to the simp-set. It changes grouping but does not identify a generally simpler form. We name it only in commands that require it.

1.6.5 6.5 Reversing an appended list

When two appended lists are reversed, the two parts occur in the opposite order. We use this lemma to prove the final property.

```
attribute [local simp] lecture07_append_empty_right

theorem lecture07_reverse_append
    α{ : Type} (xs ys : CustomList α) :
```

```
    reverse (append xs ys) = append (reverse ys) (reverse xs) := by
  induction xs with
  | empty =>
      simp [append, reverse]
  | cons x xs ih =>
      simp [append, reverse, ih, lecture07_append_assoc]
```

In the `empty` case, `simp` automatically uses the local theorem about appending the empty list. In the `cons` case, the induction hypothesis handles the tail, while `lecture07_append_assoc` groups the append operations in the same way on both sides.

The theorem about reversing an appended list is also useful as an automatic rule in the final proof, so we add it to the local simp-set:

```
attribute [local simp] lecture07_reverse_append
```

1.6.6 6.6 Reversing twice

We can now prove the original property. The induction follows the two constructors of `CustomList α`. In the `cons` case, the previous lemma describes the reversal of the append operation produced by the algorithm, and the induction hypothesis simplifies the double reversal of the tail.

```
theorem lecture07_reverse_reverse
    α{ : Type} (xs : CustomList α) :
    reverse (reverse xs) = xs := by
  induction xs with
  | empty =>
      simp [reverse]
  | cons x xs ih =>
      simp [reverse, ih, append,
        lecture07_append_empty_right,
        lecture07_reverse_append]
```

This example connects three levels. The constructors `empty` and `cons` describe the data structure. `append` and `reverse` are recursive programs over that structure. The theorems state expected properties, and the kernel checks that their proofs cover every input.

The theorem is meaningful, but it is not a complete specification of `reverse`. A function that always returned its input would also satisfy the double-reversal equation. A fuller account of the algorithm needs further properties, such as the theorem connecting `reverse` with `append`, or a property describing how element positions change.

1.7 Working strategy

1. Identify the constructors of the type. 2. Read which argument is analysed by the definition. 3. Check whether the goal reduces with `rfl`. 4. Use previously proved

equalities with `rw`. 5. If a variable remains in the analysed argument, choose that variable for induction. 6. Automate with `simp only` or `simp`. 7. When verifying a program, state the expected property first and introduce only the auxiliary lemmas required by the proof.

1.8 Sources

- Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich, *Theorem Proving in Lean 4*, "Inductive Types": https://lean-lang.org/theorem_proving_in_lean4/Inductive-Types/.
- Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich, *Theorem Proving in Lean 4*, "Induction and Recursion": https://lean-lang.org/theorem_proving_in_lean4/Induction-and-Recursion/.
- Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich, *Theorem Proving in Lean 4*, "Tactics": https://lean-lang.org/theorem_proving_in_lean4/Tactics/.
- David Thrane Christiansen, *Functional Programming in Lean*, "Datatypes and Patterns": https://lean-lang.org/functional_programming_in_lean/Getting-to-Know-Lean/Datatypes-and-Patterns/.
- Lean project, *The Lean Language Reference*, "Recursive Definitions": https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/.
- Kevin Buzzard and Jon Eugster, *Natural Number Game*: https://adam.math.hhu.de/.
- Vrije Universiteit Amsterdam, *Logical Verification*: https://research.vu.nl/en/courses/logical-verification-5/.
- Marieke Huisman and Anton J. Wijs, *Concise Guide to Software Verification: From Model Checking to Annotation Checking*: https://research.utwente.nl/en/publications/concise-guide-to-software-verification-from-model-checking-to-ann/.
- Hana Chockler, Joseph Y. Halpern, and Orna Kupferman, *What Causes a System to Satisfy a Specification?*: https://arxiv.org/abs/cs/0312036.